Does more retrieved context reduce unsupported claims?

DSAI 4207 Introduction to Large Language Models — Course Project Proposal

Concept Note Format

2–3 pages (excluding references)

All orange text in this file is temporary guidance and can be deleted/replaced in your final version.

How to use this template: *Write in full paragraphs, not checklist bullets. Use this as structure only. Your score should come from the strength of your idea and argument, not from how many fields are filled. Your project should be about a large language model (or a component of one) and must be testable with the compute you actually have; a small, well-controlled experiment beats an ambitious one you cannot finish.*

Project and Team Details

Team members	*Vsevolod Kevrukh (25131691D), Choi Wai Lun (25056276D), Ng Jun[25135706D]*
Main contact	Vsevolod - 25131691d@connect.polyu.hk, *Choi Wai Lun - 25056276D*@connect.polyu.hk, *Ng Jun- 25135706D*@connect.polyu.hk
Project type	*(C) Prompting, RAG and agents. The scored core is the evaluation, not an interface.*
One-line thesis	*On one fixed course-material folder, the same API model makes fewer unsupported claims as retrieved chunks rise from 0 to 4; more chunks after that need not help.*

1. Problem Statement and Objective

Suggested length: about 250–350 words (minimum two full paragraphs)

Retrieval-augmented generation is often treated as a fix for unsupported answers: give the model passages and it should stop inventing. That claim is only half specified. A single comparison of with-retrieval against without-retrieval hides the amount of context, and a comparison against a stronger commercial model mixes retrieval with model size.

The behaviour we study is therefore narrow. We ask how the rate of unsupported claims changes when the same model sees more retrieved chunks from a fixed folder, and nothing else changes. The research question is: on questions written from a fixed folder of DSAI 4207 lecture notes or a short FAQ, does the unsupported-claim rate of one API model fall as the number of retrieved chunks goes from 0 to 1, 2, 4, and 8? The objective is to build that small program and measure the curve, including cases where extra chunks make the answer worse. Falsifiable hypothesis: the unsupported-claim rate at 4 chunks is at least 15 percentage points lower than at 0 chunks. If the rate is flat, or if it rises after 2 or 4 chunks, the claim that more retrieved context always helps is rejected.

2. Related Work and Baselines

Suggested length: about 150–220 words

Lewis et al. (2020) introduced retrieval-augmented generation for knowledge-intensive tasks. Gao et al. (2023) survey the later pipeline of chunking, embedding, and generation. Ji et al. (2023) separate hallucination from other generation errors, which is why we mark claims rather than score a whole answer as right or wrong. Liu et al. (2024) show that models use the middle of a long context poorly, and Shi et al. (2023) show that irrelevant context can hurt question answering. Those results motivate a curve, not a single on/off switch. The required baseline is the same API model on the same questions with zero retrieved chunks. It is fair because the model, the prompt family, the questions, and the decoding settings stay fixed; only the number of chunks changes. A second hosted model may be logged as an outside column, but it is not a fair baseline for this question. What is new for the course is the claim-level curve on our own folder, plus a short failure analysis of missed chunks, wrong chunks, and ignored text. We are not proposing a new retriever.

3. Proposed Method

Suggested length: about 300–450 words (minimum two full paragraphs)

The system sits in the prompting and retrieval stage. We do not train a model. We download a small folder once: lecture notes from DSAI 4207, or a short FAQ if the notes are too thin. Pages are saved locally and cut into chunks of roughly 150 to 250 words. Chunks are embedded with a small open embedding model and stored for local similarity search. At answer time there is no web search. A run takes a question, retrieves the top k chunks, and sends them to one hosted API model with a fixed prompt: answer only from the passed chunks, quote the supporting sentence, and say that it does not know if the chunks do not support the answer. At k = 0 the same prompt is used with no chunks, so the model answers from its weights. That empty run is the baseline, not a broken system. The only factor we vary is k, in the set {0, 1, 2, 4, 8}. Temperature is 0.

The decisive experiment is those five runs on the frozen questions. If the unsupported-claim rate falls from 0 to 4 chunks, the hypothesis holds. If it is flat, retrieval did not change this model on this folder. If it falls and then rises, extra chunks added noise, which is still a result. We do not assume the curve is a parabola, and we will not draw the shape before the runs.

4. Data, Models, Setup, and Evaluation

Suggested length: about 180–260 words

The collection is the course folder itself, not a public benchmark. We will write 30 to 40 questions whose answers are in that folder, and freeze the questions before the runs. There is no train split. Marking is done by the team against the chunks that were actually passed, using a short written rule: a sentence is supported, unsupported, or an abstention. The primary metric is the unsupported-claim rate. We also record the support rate and the abstention rate. Generation uses one hosted API model, the same model for every k, at temperature 0, with one run per question. Embeddings run locally on CPU. Course notes may already be in the model's training data, so a strong k = 0 score would not by itself prove retrieval. That is why support is marked against the passed chunks, not against the model's private memory. The budget is about 200 API calls, plus local CPU for embeddings. No GPU is required. A second hosted model such as GPT or Grok may be logged as an extra column, but it is not the comparison that answers the question. Risk: the API budget or rate limit fails. Fallback: cut to 20 questions, or switch the same protocol to a small open-weight model on Colab.

5. Timeline and Roles

Week	Milestone	Owner(s)
Week 8–9	*Folder downloaded, 30-40 questions frozen, k=0 baseline running.*	*Vsevolod; Wai Lun; Ng Jun*
Week 10–11	*Search and five runs (k = 0, 1, 2, 4, 8) logged.*	*Vsevolod; Wai Lun; Ng Jun*
Week 12	*Sentence marks, graph, and 5-10 failure notes.*	*Vsevolod; Wai Lun; Ng Jun*
Week 13	*Report, code, and presentation rehearsal.*	*Vsevolod; Wai Lun; Ng Jun*

Kevrukh owns the folder and the frozen questions. One teammate, named when the team sheet is filled, owns the Python program: chunking, embeddings, search, the API call, and the five logged runs. The other owns the marking rules, the table, the graph, and the failure notes. Names and student IDs of the two teammates are still blank and will be added before submission.

6. Responsible Use and Tool Disclosure

Suggested length: about 2–4 sentences

Course notes will be used only for this assignment and will not be republished. The API model will be called under its provider terms. The folder contains no student personal data and no patient data. Coding assistance in Cursor may be used for the pipeline; prompts and generated code will be understood and disclosed. All runs, marks, and the written analysis will be the team's own work. AI-generated experimental numbers will not be fabricated.

References

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *arXiv:2005.11401.*

Gao, Y., Xiong, Y., Gao, X., et al. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv:2312.10997.*

Ji, Z., Lee, N., Frieske, R., et al. (2023). Survey of hallucination in natural language generation. ACM Computing Surveys.

Liu, N. F., Lin, K., Hewitt, J., et al. (2024). Lost in the middle: How language models use long contexts. Transactions of the ACL.

Shi, F., Chen, X., Misra, K., et al. (2023). Large language models can be easily distracted by irrelevant context. ICML.